

Water-monitor pipeline

How a drop of water becomes a number on the dashboard — from ESP32 pulse counting, through event

detection, artifact screening, fixture labeling, and finally aggregation into totals.

Every step below names the responsible **file/function**, the **gates with their real thresholds**, the **DB columns written**, and an **"if it breaks here"** symptom so you can jump from a wrong number straight to the code that produced it.

Verified against commit d81b7e3 — **0.3.1-dev31 + firmware 3.13.2**. This revision rewrote flow measurement (`pulse_meter`), replaced the dribble verdict with `below_meter_floor`, widened signatures to 256 points, and added a booster-pump subsystem (Part 5). If you touch the pipeline, re-check the constants cited here against the code before trusting them.

Two invariants hold the whole thing together:

- **The leak alarm is firmware-side and reads live flow.** Nothing in the database — no split, zero, reprocess, or relabel — can mask a real leak.
- **Every litre that reaches a total passes through `apply_effective_volume()` (step 12).** The four feedback loops (hygiene auto-split, meter audit, your labels, recompute) all re-enter there rather than writing totals directly. If totals ever disagree with the sum of events, that function — or a caller that bypassed it — is the first suspect.

The whole pipeline at a glance

[pipeline overview diagram — see the flowchart in PIPELINE.md / the repo]

Part 1 — Capture (raw water → stored event)

1 · ESP32 senses

``firmware/esp-water-shut-off-3_13.yaml`` (firmware 3.13 rewrote flow measurement)

- **Flow chain:** ESPHome ``pulse_meter`` — times the interval between pulse edges (100 μ s bounce filter), so the rate is already period-smoothed; no fixed sampling window. Rate (pulses/min) $\div$ `pp1_main / pp1_irr` (runtime NVS number entities) $\rightarrow$ L/min, **no $\div 60$** (`pulse_meter` already reports per-minute). Clamp >200 and <0.01 L/min to 0; published as `water_usage1 / water_usage2` throttled to 4 Hz (`throttle: 250ms`).
- **Volume from pulse totals, not rate integration:** litres accumulate from the `pulse_meter` total's count deltas (`water_total_1_*`), not by integrating the rate — integrating would overcount every tail by `last_rate  $\times$  timeout`.
- **``timeout: 10s`` + fast-zero tail:** `pulse_meter` holds its last rate until the 10 s timeout, so a 250 ms interval publishes 0 early (once $\sim 2.5\times$ the expected pulse interval passes, floored at 600 ms) to kill the flat "square tail" a closed tap would otherwise leave.
- **Pressure chain:** ADC two-point calibration $\rightarrow$ `pressure_main / pressure_irr` at 1.375 s window, 2 Hz (**recorded by HA** — this is all backfill ever sees) and `pressure_*_fast` at 50 ms, 40 Hz (**live only, never recorded**).
- **Onset flag:** `flow_pulse_onset_*` turns ON at any pulse within a 1 s window, `delayed_off: 8s` so

brief pauses don't split a draw (now keyed off last-pulse age, same behaviour).

If it breaks here: wrong PPL scales every volume by a constant factor; no pulses means nothing downstream ever fires. Check the HA water_usage sensor against a bucket test and the pp1_* number entities. A flat non-zero "tail" after a tap closes → the fast-zero interval isn't firing.

Branch · Leak alarm (dead end by design) trickle_alert_main / trickle_alert_irr fire when flow stays inside the min-max band for the threshold duration → ☐ alert → optional valve auto-close. Resets on any out-of-band flow. **Fully independent of the database** — it reads live flow, so no split/zero/reprocess/relabel below can ever mask a leak.

2 · Home Assistant relays

``app/ha_client.py · subscribe_entities``

A persistent websocket dispatches flow, fast pressure, and onset states to one `CircuitEventDetector` per circuit. HA's recorder separately stores 2 Hz pressure, onset transitions, flow, and the cumulative meter — the raw material replayed later by the importer (step 6) and the meter audit (step 12).

If it breaks here: an add-on restart wipes the in-progress event, so draws spanning a restart come back short or missing and are recovered only by the backfill importer. Look for restart timestamps in the add-on log near the gap.

3 · Does an event start?

``app/event_detector.py · CircuitEventDetector`` — first trigger to fire wins.

Trigger	Condition
Flow	flow $\geq$ MIN_FLOW_LPM (per-circuit floor = 60 ÷ PPL) sustained FLOW_START_SECONDS = 2.0, tolerating 2 sub-floor samples mid-ramp (FLOW_START_DIP_TOLERANCE)
Pressure	40 Hz fast-stream drop $\geq$ threshold, but only after the settled baseline has been stable $\geq$ PRESSURE_STABLE_DURATION_S = 10 (blocks oscillation peaks)
Pressure + flow	pressure opens the event first; flow arrival enriches it. start_trigger records which fired.

Stale-timer guard (dev29). If the flow-sustain timer is armed but $>$ FLOW_START_STALE_GAP_S = 30 passes with no flow sample, the timer is discarded and the next sample starts fresh. Without it a timer armed by a brief slug could stay armed for minutes (the pulse_meter rate simply stops ticking rather than sending zeros), so the next burst instantly satisfied the 2 s sustain and **backdated `start\_ts` across the whole quiet gap** — the bug that merged booster-pump top-up slugs ~5 min apart into one inflated event.

If it breaks here: missed small draws mean the floor is too high for the meter; phantom starts mean pressure oscillation is passing the 10 s stability gate; separate draws merged into one long backdated event → the stale-timer guard. Compare event start times against raw water_usage history.

4 · While the event is open

``app/event_detector.py · RawEvent``

Two flow records accumulate in parallel: a 1 Hz flow_readings list (→ the 256-point signature) and timestamped flow_samples (→ the volume integral). A 400-sample pressure ring buffer (10 s @ 40 Hz) feeds the settled-baseline tracker (a 2 s window, 3 s back). The **propagation scan** searches up to 12 s before flow onset (_PROP_MAX_LOOKBACK_S), using a 0.10 psi noise band and 5 consecutive at-baseline samples to confirm the transient onset → propagation_delay_ms.

Branch · ESP full-resolution waveform `_wf_*` constants. Firmware can stream full-resolution waveform chunks (≤ 1500 samples each, ≤ 30 records per circuit, 2 h TTL). If assembly succeeds, `signature_source = 'esp_full_*`'; otherwise the software 1 Hz signature is the fallback.

5 · How does the event end?

``app/event_detector.py`` — any one of five exits closes it.

Exit	Condition
Normal	onset OFF and flow < MIN_FLOW_LPM, after a ≤ 120 s low-flow grace (LOWFLOW_OFF_GRACE_S) that coalesces fragments into one draw
Pressure recovery	dip recovered to $\leq 50\%$ of its magnitude (PRESSURE_RECOVERY_FRACTION) for 10 s
Flow override	pressure back at baseline for 5 min (PRESSURE_RECOVERY_FLOW_OVERRIDE_S) → close even if flow reads stale-high (the cause of the old 27.6 h irrigation event)
Settled no-flow	flow zero the whole event + pressure settled for 60 s (SETTLED_NOFLOW_CLOSE_S) → close (a stuck event blocks every new event on the circuit)
Watchdog	hard force-close at 6 h (MAX_EVENT_DURATION_S)

If it breaks here: two draws merged into one points at the 120 s coalesce grace; a stretch of missed events points at something sitting open — grep the log for force-close / settled-no-flow lines.

Branch · Discarded (never becomes a row) Volume < 1 mL (MIN_EVENT_VOLUME_L) is noise. A pressure-surge phantom — max pressure > baseline + 0.5 psi (PRESSURE_SURGE_PHANTOM_PSI) with no net drop — is a turbine artifact. Both are dropped.

6 · Features computed

``app/feature_extractor.py`` · `FeatureExtractor`

- **Volume:** time integral of the timestamped samples (`flow_integral.integrate_litres`). A gap > 300 s marks `integration_quality = 'degraded'` (kept out of training).
- **Signatures:** 256-point proportional flow + pressure envelopes (`SIGNATURE_POINTS = 256`, widened 32→64→256; point $i = i/256$ through the event) — plus a separate 32-cell $\times$ 1 s absolute-time onset/offset **edge signature** (`onset_signature_json / offset_signature_json`) that feeds the edge-signature k-NN tier.
- **Hydraulics:** `pressure_delta_psi`, `pre_event_pressure_psi`, propagation delay, resistance shape.
- **Active-flow metrics** the verdicts depend on: `flow_integral_litres`, `flow_on_ratio`, `true_avg_flow_lpm`, `active_flow_segment_count`.
- **Time features:** `hour_sin / hour_cos`, day-of-week, weekend.

If it breaks here: event volume disagreeing with the meter → check `integration_quality` (gap 'degraded') and `volume_recorder_litres` for the firmware-meter cross-check.

Branch · Backfill importer joins here ``app/historical_importer.py``

- **Triggers:** startup (back to `MAX_BACKFILL_DAYS`), periodic catch-up from `import_state.last_check_ts`, or a manual range.
- **Sources:** `flow_pulse_onset` ON/OFF transitions first (bridging gaps < 15 s of turbine chatter), then a sustained-flow-threshold fallback.
- **Duplicate gate:** a period overlapping an existing event by ≥ 30 s (or ≥ 10 s **and** $\geq 80\%$ of the

shorter) is skipped — safe to re-run.

- **Irrigation cross-talk post-pass:** flags main-circuit events overlapping irrigation activity when the pressure-swing ratio (irrigation $\Delta \div$ main Δ) ≥ 1.3 , with the frozen ≤ 1.5 L cap $\rightarrow$ `match_rejection_reason = 'irrigation_cross_talk'`.
- Rebuilt events always carry `start_trigger = 'flow'` (no 40 Hz pressure history exists to reconstruct).

If it breaks here: duplicated or still-missing backfill $\rightarrow$ check `import_state` and whether the gap's HA recorder history actually exists.

Part 2 — The verdict cascade (is it real water?)

``app/feature_extractor.py · _finalize_derived_verdicts()`` — checked strictly in this order; the first match wins and sets the effective volume.

#	Check	Condition	Result
7a	You already classified it	<code>user_classified = 1</code>	your verdict stands; auto-detection never re-flags
7b	Durable irrigation cross-talk	<code>match_rejection_reason = 'irrigation_cross_talk'</code> (from the importer)	<code>veff = 0</code> ; survives every reprocess unless you relabel it real
7c	Durable overlap-duplicate (dev28)	<code>mrr = 'overlap_duplicate'</code> already stamped out-of-band (an event the overlap guard found duplicating another)	joins phantom flag, <code>veff = 0</code> ; preserved through reprocess
7d	Phantom (pressure restoration)	<code>duration ≥ 120 s</code> (<code>_PHANTOM_NOFLOW_MIN_DURATION_S</code>); a legacy event with no active-flow metrics needs the frozen 30-min floor instead) $\wedge \Delta P < 2.0$ psi (frozen — leak safety) $\wedge$ <code>flow_integral < 1.0 L</code> $\wedge$ <code>flow_on_ratio < 0.05</code> . Rescue: true avg flow ≥ 2.0 L/min $\rightarrow$ real brief draw, not phantom	<code>is_pressure_restoration_phantom = 1</code> , <code>veff = 0</code> , <code>mrr = 'pressure_restoration_phantom'</code> , excluded
7e	Pump recharge (dev24, pump mode only)	fires only when pump mode is active (see Part 5). A booster-pump top-up slug: <code>0 < vol ≤ 0.6 L</code> (<code>PUMP_SLUG_MAX_L</code>) $\wedge$ <code>0 < dur ≤ 60 s</code> (<code>_PUMP_SLUG_MAX_DURATION_S</code>) $\wedge$ (<code>flow_pressure_corr ≥ 0.5</code> or <code> \Delta P ≤ 0.8 psi</code>). Runs ahead of 7f/7i and replaces them under a pump sawtooth	joins phantom flag, <code>veff = 0</code> , <code>mrr = 'pump_recharge'</code> , excluded
7f	Rising-pressure phantom (dev14)	the opposite of 7d — flow tracked a pressure rise (turbine spun on climbing supply pressure, not demand). <code>flow_pressure_corr ≥ 0.6</code> (<code>_RISE_PHANTOM_MIN_CORR</code>) $\wedge$ <code>0 < vol < 1.0 L</code> turbine / <code>2.5 L</code> positive-displacement (frozen leak guard) $\wedge$ <code>dur ≤ 120 s</code> . <code>corr = None</code> $\rightarrow$ never fires; waived in pump mode	joins phantom flag, <code>veff = 0</code> , <code>mrr = 'rising_pressure_phantom'</code> , excluded
7g	Cross-talk (other circuit's draw)	same no-flow ceilings (<code>flow_integral < 1.0 L</code> , <code>flow_on_ratio < 0.05</code>), but a real drop: <code>$\Delta P \geq 2.0$ psi</code> $\wedge$ <code>duration ≥ 120 s</code> (<code>_XTALK_MIN_DURATION_S</code>). The ΔP floor separates it from 7d	<code>is_cross_talk = 1</code> , <code>veff = 0</code> , excluded

#	Check	Condition	Result
7h	Below-meter-floor (was "dribble")	the flow never rose enough for the meter to register reliably: the event's active flow ($\text{true_avg_flow_lpm}/\text{peak_flow_lpm}$, not the zero-diluted whole-event average) < the meter's registration floor — 1.1 L/min positive-displacement / 1.0 L/min turbine ($\text{_METER_FLOOR_PD_LPM} / \text{_METER_FLOOR_TURBINE_LPM}$; PD class chosen when $60 \times \text{PPL} \geq 0.5$). Volume, ΔP and duration are not gates. One valid-regime burst vetoes it	$\text{is_low_flow_dribble} = 1$ (flag reused), $\text{veff} = 0$, $\text{mrr} = \text{'below_meter_floor'}$, excluded; bidirectional reprocess
7i	Pressure-silent flow (registration-floor doctrine)	valid flow (active metric $\geq$ floor) but no pressure response: $\Delta P < 0.8$ psi ($\text{_PSILENT_MAX_DELTA_PSI}$) $\wedge$ $\text{flow_pressure_corr}$ present and < 0.3 $\wedge$ no pressure transient $\wedge \text{dur} \leq 300$ s $\wedge 0 < \text{vol} \leq 5$ L. $\text{corr} = \text{None} \rightarrow$ never fires; waived in pump mode	joins phantom flag, $\text{veff} = 0$, $\text{mrr} = \text{'pressure_silent_flow'}$, excluded
7j	Degraded supply (pulsing water)	$\text{degraded_supply} = 1$ — flow reading unreliable	$\text{veff} = \text{volume_litres_estimated}$, capped (see below), method $\text{'pulsing_supply_envelope'}$, excluded
7k	Sparse envelope (brief use, long idle tail)	$\text{duration} \geq 10$ min ($\text{_SPARSE_ENVELOPE_MIN_DURATION_S} = 600$ s) $\wedge$ flow on $\leq 10\%$ of it	litres kept , $\text{mrr} = \text{'sparse_envelope'}$, excluded from training; targeted by the hygiene loop
8	Real use	none of the above	$\text{volume_litres_effective} = \text{volume_litres}$, method 'raw' , eligible for training + totals

The zeroing verdicts 7d–7i all set $\text{is_pressure_restoration_phantom}$ (or is_cross_talk) — they share the flag/hide-toggle plumbing and differ only by $\text{match_rejection_reason}$, which carries the true provenance.

Below-meter-floor replaced the old dribble verdict ('a719e85'). The dribble rule zeroed on volume/flow/ ΔP thresholds; the registration-floor rule instead asks the one physical question that matters — did flow ever rise enough for this meter to measure it? The old _DRIBBLE_ * constants still exist as dormant calib keys but the detector no longer reads them. Its reprocess pass is **bidirectional**: it flags newly-sub-floor rows and restores rows the old triple-gate wrongly zeroed.

Envelope cap (dev10, guards 7j). A degraded/pulsing envelope estimate can badly over-read, so $\text{_cap_envelope_estimate}()$ limits it to $\max(1.5 \times \text{flow_integral_litres}, 2.0 \text{ L})$ ($\text{_ENVELOPE_CAP_FLOW_MULT} = 1.5$, $\text{_ENVELOPE_CAP_FLOOR_L} = 2.0$); the uncapped value + cap base go to $\text{degraded_diagnostic_json}$ for audit.

Suppression-averted backstop (dev10, wraps 7d). A leak-safety guard: when the phantom rule would fire but the event actually **measured** $\text{'volume_litres} \geq 10 \text{ L}'$ ($\text{_PHANTOM_REVIEW_FLAG_LITRES} = 10.0$), the phantom verdict is averted — the volume is **kept, not zeroed** — and the event is surfaced for review ($\text{phantom_suppression_averted} = 1$, $\text{anomaly_type} = \text{'suppression_averted'}$, $\text{flagged} = 1$; still $\text{excluded_from_training}$). Rationale: silently zeroing 10 L+ is riskier than showing a "please review" draw. Migration 20260551 restored already-zeroed large phantoms through the ledger.

Where $\text{'flow_pressure_corr'}$ comes from (7f/7i). The correlation is computed live from the event's waveform ($\text{_flow_pressure_correlation}$ over the full flow + pressure series) and stored on the row (column added by migration 20260554). Stored signatures **cannot** reconstruct it — the pressure signature clamps drop-fraction to [0, 1], erasing the above-baseline rises the discriminator depends on — so historical events are handled by a

one-shot backfill worker (see the branch below). The verdict is (re-)applied to any event carrying a stored corr by `reprocess_rising_pressure_phantoms()`, part of the same repair chain as the other artifact passes.

If it breaks here: real water zeroed (or noise surviving to step 8) → read `match_rejection_reason` first — it names the exact verdict (`below_meter_floor`, `pressure_silent_flow`, `pump_recharge`, `rising_pressure_phantom`, ...) — then `volume_estimation_method` and the `is_*` flags. A big draw flagged rather than counted-clean → `phantom_suppression_averted`. Relabeling the event re-runs the cascade with your label winning.

Branch · Rising-corr backfill (dev14, one-shot) ``app/rise_corr_backfill.py``

Events predating the `flow_pressure_corr` column can't have it re-derived from their stored signatures, so this worker computes it the trustworthy way: it re-fetches each candidate's flow + pressure history from the HA recorder, resamples both onto the importer's 1 Hz grid, runs the same `_flow_pressure_correlation`, then applies the verdict through the shared repair pass (one ledger path). **Candidates** are only events the live detector could have caught: ≤ 120 s, $0 < \text{vol} < 1$ L, unlabeled, unflagged, non-degraded, no artifact verdict, no stored corr. It sweeps oldest-first; a clean sweep sets `home_profile.rise_corr_backfill_done = 1` and never runs again (events whose history is gone stay counted — leak-safe default).

9 · Row stored + hour ledger posted

``app/database.py · upsert_event_and_apply_hourly_volume()``

One row in events (volumes raw + effective, signatures, artifact flags, provenance columns) is written, and the event's litres are posted to its hour bucket via the chokepoint (step 12) in the same step. Sequence context (`seconds_since_prev_event`, `cycle_pulse_count`) is filled in.

Branch · Hygiene auto-split loop ``app/reprocess.py``

- **Candidates:** unlabeled events whose actual flow is $< 25\%$ of their span (including inflated `sparse_envelope` singles).
- **Dry-run:** the importer counts real draws inside; if 2–10 are found →
- **Atomic ``reprocess_window()``:** delete → auto-widen to engulf the deleted spans → re-import → restore the originals on any failure (all-or-nothing, no lost water).
- **Never touches** user-labeled, artifact-flagged, anomaly-flagged, or softener-session events.
- Loops back through step 6.

Part 3 — Labeling (which fixture?)

``app/database.py · reclassify_all_events_from_signatures()`` — a ladder; the first rung to claim an event stamps `matched_fixture_type + matched_via` and the event exits. Cycle/session fixtures also get a `cycle_group_id` (the History rollout key).

Run g	Detector	Gates	Stamps
10a	Softener session (<code>detect_softener_sessions</code>)	skipped unless has <code>water_softener</code> + right circuit. Low-flow draws (≤ 1.5 L/min, peaks < 4) at regen time ± 20 min (local clock); chains gaps ≤ 45 min, session ≤ 210 min; needs a backwash ≥ 30 L and brine span ≥ 90 min	<code>water_softener</code> , <code>matched_via='softener_session'</code>
10b	Washer cycle (<code>detect_washer_cycles</code>)	anchor fill ≥ 9 L, 80–400 s, peak 7.5–15 L/min; siblings at 0.8–1.3× anchor peak, 2–45 min away, ≥ 0.5 L, ≤ 400 s, not flush-shaped; needs anchor + ≥ 2 siblings. Live path retro-stamps mates from the trailing 50 min	<code>washing_machine</code> , <code>matched_via='washer_cycle'</code>

Run g	Detector	Gates	Stamps
10c	Dishwasher cycle (detect_dishwasher_cycles)	≥3 chained gentle fills: 0.2–3.5 L, peak ≤3.6 L/min, not flush-shaped; consecutive fills ≤30 min apart, whole run ≤180 min; skips artifacts + washer/softener members	dishwasher, matched_via='dishwasher_cycle'
10d	Per-event rules (rule_classify_event)	first hit wins — see below	fixture type, matched_via='rule_*/' / 'zone_default'
10e	Fingerprint tier (fingerprint_matcher.py)	only if rules abstain, before k-NN; now with a 2 L floor — see below	fixture type, matched_via='fingerprint'
10f	k-NN residual	only if the fingerprint tier also abstains. Internally a 3-level sub-ladder: edge-signature → plain active-flow → legacy 6-scalar — see below	fixture type, matched_via='knn' (all three sub-levels)
10g	Composite (composite_detector.py)	sustained ≥300 s + usable waveform (≥30 bins, ≤15 s/bin); embedded toilet = 3–8 L excess at ≥3 L/min over a rolling 35th-percentile baseline	promotes unlabeled → other, matched_via='composite'; writes embedded_fixtures_json

****Toilet physics veto (dev17) — post-filter on every toilet label**** (from rule, fingerprint, or k-NN).

toilet_physics_veto() turns a proposed toilet into an abstention (never re-guessed) when any of: volume < 2.8 L (TOILET_MIN_FLUSH_L, below the smallest flush ever made); volume > the era cap (toilet_flush_cap_litres from home_profile.build_year — 1994+ ≈ 7.0 L, 1982–93 ≈ 13.2 L, else ≈ 30.5 L, each ×1.15); peak < 3.0 L/min (TOILET_VETO_MIN_PK_LPM); or > 2 active flow segments (a cistern refill is one continuous segment). These are structural constants, never per-home calibrated.

10d · Per-event rules (first hit wins):

- **Toilet:** 2.2–8.5 L, 20–150 s, peak ≥5 L/min, **and** (pressure transient or $\Delta P \geq 1.5$ psi).
- **Dishwasher (single):** 0.2–3.5 L, peak ≤3.6 L/min, cycle_pulse_count ≥3.
- **Shower:** ≥30 L ∧ ≥300 s ∧ peak ≥6 L/min — or 15–30 L ∧ ≥240 s.
- **Zone (zone circuits only):** ≥240 s ∧ peak ≥5 L/min.

Bands are per-home calibrated once at activation (rule_calibration.py: weighted-percentile fit, capped at ≤2× the default span, do-no-harm k-fold validation — a fit that regresses recall is discarded and the default kept). A PPL change triggers partial recalibration of artifact thresholds only, never these rule bands.

10e · Fingerprint tier (dev11, fingerprint_matcher.py):

A whole-waveform nearest-neighbour match — much stronger evidence than k-NN's scalar summaries, so it runs ahead of k-NN and short-circuits it on a hit.

- **Fingerprint:** the event's un-normalised waveform trio — absolute-time flow (L/min), cumulative volume (L), and pressure drop below baseline (psi) — sampled on a 4 s grid, 64 cells (first 256 s). Built only when the waveform supports it: ≥10 s, ≥4 bins, peak ≥0.3 L/min.
- **Library: user-labeled events only** (user/training/cycle sources — all carry user_fixture_type). Fingerprint labels are never added to the library, so there is no fingerprint→fingerprint chaining and no drift.
- **Gates:** ≥10 library labels total (MIN_LIBRARY_N), ≥5 per class (MIN_CLASS_LIBRARY), and a

2 L effective-volume floor (MIN_MATCH_VOLUME_L = 2.0, dev20) on both library membership and matching — since firmware 3.13 eventizes sub-2 L micro-draws, a fresh-DB review overturned every sub-2 L fingerprint stamp, so

those now abstain. The accept distance is **self-calibrating** — a percentile of the library's own nearest-neighbour distance distribution, recomputed at load: 30th percentile once mature (≥ 100 labels, `THRESHOLD_PCTL_MATURE`), tightened to the 15th below that.

- **CYCLE_ONLY exception:** unlike k-NN, this tier may inherit `washing_machine / dishwasher` — a full-waveform match against a user-confirmed example is trusted (measured 83/83 dishwashers correct).
- Measured on this home: ~30% coverage at ~97% precision; ~29% of the events the rest of the pipeline declined, at ~94%. Per-circuit 5-min library cache, invalidated when you save a label.

10f · k-NN residual — a 3-level sub-ladder, all stamping `matched_via='knn'`:

- **Level 1 · edge-signature k-NN (dev19):** runs when the event and its neighbours all carry decodable 32-cell onset/offset edge signatures. Adds 64 edge features (`onset_00..offset_31`, per-dim scale 1.0) on top of the active-flow block, so it matches on the shape of the open/close edges, not just scalars. If it abstains it falls **straight to the legacy fallback** (level 3) — deliberately not back to level 2. On disk it's indistinguishable from the other levels (all `knn`; internal `match_source = 'active_flow_edges'` isn't persisted).
- **Level 2 · plain active-flow k-NN:** log-scaled features weighted volume 1.52, ΔP 2.88, duration 1.40, flows 0.70/0.74, `flow_on_ratio` 0.25, `cycle_pulse_count` 0.75, `hour_sin/hour_cos` 0.35.
- **Level 3 · legacy 6-scalar fallback** for pre-backfill events.
- **Vote (all levels):** $k=5$, inverse-distance weighted, ≤ 4 neighbours per class. **Abstains when:** total score < 1.5 , winner's share < 0.6 , or the winner is other.
- **CYCLE_ONLY guard:** a lone k-NN vote can never stamp `washing_machine / dishwasher / water_softener` — only their cycle detectors (or the fingerprint tier above) may.

If it breaks here: a wrong fixture name → read `matched_via` first; it names the exact rung that claimed the event, so you know which thresholds to compare against the event's volume / duration / peak.

11 · Anomaly score + surfacing (every event)

``app/anomaly_baseline.py``, ``app/alert_manager.py``, ``app/routers/history.py``

Scored against the frozen per-home baseline (fit at activation, never online-adapted): volume, type, and time-of-day pattern → `anomaly_score / anomaly_type`, setting `flagged = 1`. The core scoring is unchanged, but dev9 wired up the surfacing that was previously dead:

- **Suppression-averted override:** a `phantom_suppression_averted` event (step 7d backstop) is forced anomalous before the artifact gate, so an excluded-from-training big draw still shows up for review.
- ``triggered_alert`` is now stamped 1 when `alert_manager.fire()` actually sends a notification — an audit trail of "this event alerted" (previously never populated).
- **History surfacing:** a `?filter=anomaly` view lists every flagged event (bypassing the "hide not-real" toggle); a display-time `anomaly_reason` splits the flag into user-facing badges — `review_draw` (suppression-averted), `estimated` (degraded/envelope), `high_usage` (rest). Marking an event reviewed sets `user_reviewed = 1` and clears it from the dashboard's unreviewed-anomaly count.
- **Review verdicts (dev13):** the review records why — `review_verdict` (column added by migration 20260553) is `normal` (confirmed legit use) or `unknown` (looked, didn't recognise it). Both stamp `user_reviewed = 1`. The verdict is **display + training-workflow only** — it never touches `volume_litres_effective` or the live (frozen-baseline) anomaly score. Its one job: `fit_usage_baselines()` **holds `unknown` events out** of the next deliberate baseline refit (`WHERE COALESCE(review_verdict, '') <> 'unknown'`), so an unidentified draw can never stretch a fixture envelope toward "normal". Labeling the event later clears an unknown verdict (identifying it

supersedes "don't know").

- **Unusual-events list fix (dev12):** the anomaly filter was applied in Python after the newest-100 fetch, so a flagged event older than the 100th row vanished from the very view meant to surface it (card said 95, list showed 2). The `flagged_only` / `degraded_only` filters now run **inside the SQL `WHERE`** in `get_recent_events()`, so the recency limit applies to matching rows and the card count agrees with the list.

Part 4 — Totals (events → the numbers you see)

12 · The volume chokepoint

``app/database.py · apply_effective_volume()``

The only path by which any event's litres reach totals:

- If a prior posting exists (`hourly_volume_applied_litres / _bucket`), post a **reverse delta** to the old hour bucket.
- If the new effective volume > 0, post it to the hour of `start_ts`. If it's 0, the bucket is NULL — the event contributes nowhere.

Callers (the complete list): live insert (step 9), `volume_recompute.py`, `recorder_reconcile.py`, the low-flow coalescer, and relabel/reprocess paths. **Invariant:** `volume_ledger_discrepancy() ≈ 0` (sum of applied amounts = sum of `hourly_volume`).

If it breaks here: totals drifting from the sum of events means a writer bypassed this function — run the discrepancy check first.

Branch · Meter audit (the reconciliation loop) ``app/recorder_reconcile.py · reconcile_circuit_volumes()``

Hourly, over settled events (> 20 min old, ≤ 6 h horizon), **healthy events only** — never any zeroing verdict (phantom family, cross-talk, below-meter-floor, degraded) or anything you classified. Computes the firmware cumulative meter's delta across the event and stores it as `volume_recorder_litres`. **Auto-corrects** only when recorder samples bracket the event edges within ±2 min **and** the divergence is > 0.5 L **and** > 20% of the larger value — routing the fix through `apply_effective_volume()` like everything else. Otherwise it just flags a backlog you can apply manually from History.

If it breaks here: per-event volumes disagreeing with the physical meter → compare `volume_litres_effective` vs `volume_recorder_litres`, then check `reconcile_state` for whether that window was ever reconciled.

13 · Hour buckets accumulate

``hourly_volume` table`

Signed deltas accumulate via `INSERT ... ON CONFLICT DO UPDATE SET volume_litres = volume_litres + excluded.volume_litres`. This table is already net of every zeroing, correction, split, and relabel above, and is what the 24 h dashboard chart reads.

Branch · Your labels (the feedback loop) ``app/routers/history.py · patch_event()``

Saving a label writes `user_fixture_type + user_classified = 1` — never overwritten by any machine pass.

Appliance labels propagate to cycle-mates (±45 min, tagged `fixture_label_source = 'cycle'`). 8 s after the last edit a background reclassify re-runs the ladder over unlabeled events (skipped once the baseline is locked). An hourly maturity recheck re-runs events < 6 h old so provisional labels settle once cycle-mates exist. Any volume change re-enters at step 12.

If it breaks here: a label that "won't stick" usually means you're looking at a different event (a split/reprocess created new IDs), or the maturity recheck re-stamped an unlabeled mate — your own labeled row is never touched.

14 · What you see

Surface	File / function	How the total is computed
Daily summary	database.py · compute_daily_summary()	SUM(COALESCE(volume_litres_effective, volume_litres, 0)) per circuit-day; phantoms excluded from the event count only (their litres are already 0)
Water Use page	database.py · get_category_rollup()	groups by effective type — precedence user_fixture_type → confirmed fixture → matched_fixture_type → cluster hint → other; WHERE is_pressure_restoration_phantom = 0; lifetime + windowed sums per fixture
Dashboard + HA	routers/dashboard.py, fixture_publisher.py	24 h chart straight from the hour buckets; per-fixture and per-category totals published to Home Assistant as total_increasing sensors
History page	routers/history.py	reads events rows directly. A filter bar (dev15) queries by date / duration / avg flow / ΔP / volume / fixture / note, pushed into get_recent_events() SQL. The standalone "Shape" column was dropped (the flow sparkline moved inline). The "hide not-real events" toggle now filters in SQL (exclude_not_real, dev22) — before dev22 it post-filtered after the 100-row limit, so a storm of pump artifacts starved the page to ~18 visible rows; count_not_real_events() backs the "N hidden — show them" badge. Totals never change either way — litres were decided at step 7

Part 5 — Booster-pump mode (dev21–27)

Homes on a booster pump behave differently from mains: the pump cycles to recharge line pressure, which would otherwise look like a storm of tiny phantom draws and trip the pressure-start trigger. The whole subsystem stays **inert until pump mode is confirmed active** — the single gate config.pump_gates_active() is true only for an active **VFD constant-pressure** profile.

- **Supply-type profile** (home_profile.supply_type, migration 20260558): mains / well / city_pump; the two pump types plus a vfd_constant_pressure profile drive everything below. pump_mode_effective() resolves per-circuit override → supply type → confirmed nightly detection; detection **never auto-enables** — a dashboard banner ("booster pump?") requires user confirmation.
- **Nightly regime detector** (pump_regime_detector.py, math in pump_regime_math.py): analyses each circuit's quiet-hour pressure/flow window, writing one row per night to pump_regime_nightly (detected, period_s, amplitude_psi, cycles, est_leak_lpd, ...). Hysteresis raises the banner.
- **Cascade effect** (Part 2): adds the pump_recharge verdict (7e); **waives** the rising-pressure phantom (7f) and pressure-silent (7i) checks, whose static-supply premise breaks under a pump; and waives the toilet rule's pressure-corroboration requirement (is_flush_shaped(pump_mode=...)).
- **Detection effect** (step 3): a live **oscillation gate** (pump_osc_gate_psi = max(2.0, 0.15 × band)) suppresses pressure-initiated starts while the rolling 60 s pressure peak-to-peak exceeds it, and surge widening relaxes the surge-phantom guard. **Flow-triggered starts are untouched**, so leak coverage is unchanged.
- **Pump-assisted leak detection** (dev26): the nightly recharge-cycle estimate (est_leak_lpd,

PUMP_SLUG_CALIBRATION_FACTOR = 1.9) drives a notify-only pump_leak alert (≥ 20 L/day over 3 nights, or $> 30\%$ week-over-week period shrink) and a dashboard leak-watch tile. Recharge volume is real water feeding a downstream leak but not fixture usage, so it stays **zeroed in usage totals**; leak accounting lives in the separate Phase-5 estimator, not the cascade.

- **Low-pressure alerts** (dev27): alert_low_pressure_supply (a zone flowing below ~ 25 PSI for 180 s) and alert_pump_low_pressure (an armed VFD home below its pump floor — distinguishes pump failure from "can't keep up"), via alert_manager.py.

Not part of this pipeline: the leak test (leak_test_scheduler.py, dev30–31, firmware 3.13.2) is a separate active valve-closed pressure-decay diagnostic with its own leak_test_history table — it is not a stage of event → label → total. It touches the pipeline at exactly one seam: the Phase-5b "Pump check" cross-reads whether the untested circuit kept recharge-cycling during a valve-closed test.

Quick debugging index

Symptom	Start at	First thing to check
A total looks wrong	Part 4	the event's volume_litres_effective vs volume_litres, then volume_estimation_method
Totals $\neq$ sum of events	step 12	volume_ledger_discrepancy() — a writer bypassed the chokepoint
Event has the wrong fixture name	Part 3	matched_via — names the exact ladder rung that claimed it
Event volume $\neq$ physical meter	step 12 branch	volume_litres_effective vs volume_recorder_litres, then reconcile_state
Event missing / merged / split	Part 1	restart timestamps (websocket), the 120 s coalesce grace, force-close lines, or the dev29 stale-timer guard (backdated merges)
Real water shows as 0 L	step 7	match_rejection_reason names the verdict, then the is_* flags + volume_estimation_method
A low draw was zeroed	step 7h	below_meter_floor — active flow never crossed the 1.0/1.1 L/min registration floor
A steady draw zeroed with no pressure dip	step 7i	pressure_silent_flow (flow_pressure_corr present & < 0.3)
Tiny draws vanish only on the pump home	step 7e / Part 5	pump_recharge — is pump mode confirmed-active when it shouldn't be?
Big draw flagged "review" not counted-clean	step 7d backstop	phantom_suppression_averted (≥ 10 L phantom kept + flagged on purpose)
Degraded volume looks capped/too low	step 7j	degraded_diagnostic_json (envelope_cap_applied, envelope_uncapped_litres)
A toilet label disappeared	step 10 veto	toilet physics veto (floor 2.8 L / era cap / peak / segments) turned it into an abstain
Label came from fingerprint and looks wrong	step 10e	2 L floor + library size / self-calibrated threshold; save a correct label to re-seed
A label won't stick	step 13 branch	whether the ID changed (reprocess) — user rows are never overwritten